

## Brain Bolt – The Engineers Sprint

### Problem Statement Template | IMECE India 2026

#### Regional Level

#### Organization Details

|                      |                          |
|----------------------|--------------------------|
| Name of Organization | Siemens Energy India Ltd |
|----------------------|--------------------------|

#### Problem Statement Details

|                               |                                                                                                |
|-------------------------------|------------------------------------------------------------------------------------------------|
| Problem Statement Title       | Battery Health Assessment and Dynamic Allocation for Light EVs                                 |
| Category / IMECE Pillar       | Energy /Manufacturing / Digitalization/Mobility                                                |
| Industry Domain               | Energy                                                                                         |
| Level of Difficulty           | Medium / Hard                                                                                  |
| Technology Recommendation     | Python, MATLAB/Simulink, Scilab, C/C++, Java or another suitable coding/simulation environment |
| Use Case (if any)             |                                                                                                |
| Literature Reference (if any) |                                                                                                |

#### Problem Statement Format

##### 1. Background:

A battery-swapping station operates a fleet of light electric vehicles. Batteries differ in state of charge, state of health, temperature, internal resistance, cycle count, cell-voltage imbalance and available energy. Vehicle requests also differ in range requirement, load, priority and maximum waiting time. Selecting batteries only by highest state of charge can lead to poor utilization, unsafe allocation and inefficient service.

##### 2. Key Objectives / Challenge:

1. Unsafe batteries must not be allocated.
2. No duplicate battery assignment.
3. No vehicle receives multiple batteries.
4. Allocated battery meets minimum acceptable SoC (State of Charge).
5. Reported metrics can be recomputed from submitted assignments.
6. Proposed method is compared against Highest-SoC-First using identical input data.

##### Data Provided for the teams:

1. Battery Fleet Dataset: 200 battery packs with health, thermal, electrical and usage parameters.
2. Vehicle Demand Dataset: 50 incoming vehicle requests with range, load, priority, minimum acceptable SoC and maximum waiting time.
3. All teams receive the same input datasets.

##### Exact problem to be solved:

Develop a coding-based or simulation-based battery assessment and allocation system that classifies batteries and assigns at most one suitable available battery to each vehicle request while satisfying the stated safety and operational constraints.

### **Mandatory Tasks:**

1. Classify every battery into exactly one category: Safe and Available; Degraded but Usable; or Unsafe / Quarantine.
2. Create a Battery Suitability Score from 0 to 100 using at least four provided battery-health parameters.
3. Allocate batteries to vehicle requests such that unsafe batteries are never allocated, each battery is allocated at most once, each vehicle receives at most one battery, and the allocated battery meets the vehicle's minimum acceptable SoC.
4. Account for vehicle priority in the allocation logic.
5. Implement the solution through actual coding or simulation.
6. Perform data analysis and visualization.
7. Quantitatively validate the proposed method.
8. Compare the proposed method against the fixed baseline: Highest-SoC-First allocation.
9. Respond to the unseen 30% onsite twist using the same core solution.

### **Mandatory Quantitative Outputs:**

1. Number of successfully served vehicles.
2. Number of unserved vehicles.
3. Percentage of High and Critical priority vehicles served.
4. Number of unsafe battery allocations.
5. Average SoH of allocated batteries.
6. Average Battery Suitability Score of allocated batteries.
7. Optional: Average waiting time, if queueing is explicitly modelled

### **Mandatory Visual Outputs:**

1. Provide at least three meaningful visualizations. Examples include:
2. Battery-health or suitability-score distribution.
3. Vehicle allocation results by priority.
4. Proposed method versus Highest-SoC-First baseline.
5. Unsafe/quarantined batteries identified by the method.

### **Fixed Baseline:**

Highest-SoC-First: Process vehicle requests in arrival order and allocate the currently available battery with the highest State of Charge that satisfies the minimum acceptable SoC. Each battery may be used only once. Teams must compare their proposed method against this baseline using the mandatory quantitative outputs.

### **Verification Rules:**

1. An unsafe/quarantined battery must never be allocated.
2. A battery cannot be assigned to more than one vehicle.
3. A vehicle cannot receive more than one battery.
4. The allocated battery must satisfy the vehicle's minimum acceptable SoC.
5. All reported results must be reproducible from the datasets provided and submitted code or simulation.

## **3. Problem Statement Split:**

### **Part 1 – 70% of the Problem Statement**

Teams receive the 200-battery dataset and 50-vehicle-request dataset and complete all mandatory tasks, outputs, visualizations, quantitative validation and baseline comparison described above.

### Important Note

The final challenge should be designed such that teams can solve, refine, and submit their solutions within a **6–8-hour time window** during the event.

### 4. Scope / Limitations (Submission Requirements):

1. Runnable code or executable simulation model.
2. Output file/table containing battery classifications and battery-to-vehicle assignments.
3. Baseline comparison.
4. Mandatory quantitative metrics.
5. At least three visualizations.
6. Short explanation of the method and response to the 30% twist.

### 5. Software & Hardware Requirements:

Mention any required software tools, hardware, platforms, or simulation environments:

- Python, MATLAB/Simulink, Scilab, C/C++, Java or another suitable coding/simulation environment.

| Requirement          | Details      |
|----------------------|--------------|
| Laptop Requirement   | Any suitable |
| Software Requirement | Listed above |
| Other Requirements   | NA           |